

Mobile Application Development with Flutter

Lecture 32. Navigation in Flutter

What is Navigation?

- Navigation means moving from one screen to another
- Navigation in Flutter is primarily managed using the **Navigator** widget and its stack-based approach
- Screens are pushed onto or popped off a stack
- For simple apps, the built-in Navigator is sufficient
- Complex apps should use a dedicated routing package like **go_router** for advanced features

Types of Navigation

- In Flutter, there are two main types of navigation:
- **Imperative Navigation**
 - Imperative navigation means you push and pop routes programmatically
 - You decide when and which route to navigate to based on user actions or other events
- **Declarative Navigation**
 - Declarative navigation allows Flutter to manage the navigation state by defining routes declaratively using Navigator 2.0
 - It is used for more complex navigation scenarios, such as apps with deep linking, custom back navigation, or nested navigation

Navigating Between Screens (Routes)

- There are two primary ways to define and manage routes:

1. Anonymous Routes

- Also called direct routes or inline routes
- An anonymous route is one defined inline when calling `Navigator.push()`
- It's a simple and quick way to navigate to a new screen

2. Named Routes

- Named routes are defined in your app's `MaterialApp` widget
- You can navigate to named routes using the `Navigator.pushNamed()` method

Comparison

Feature	Anonymous Routes	Named Routes
Setup	No setup needed	Defined in MaterialApp
Code	Inline	Centralized
Use	Small apps	Large apps

Core Navigation Concepts

- Navigator
 - A widget that manages a stack of Route objects
- Route
 - In Flutter, a screen or page is referred to as a route
- Push()
 - Adds a new route to the top of the stack, making it the current active screen
- Pop()
 - Removes the top-most route from the stack, revealing the previous screen

What is pushReplacement?

- Go to a new screen AND remove the current screen from the stack
- Meaning the user cannot navigate back to the previous screen using the back button
- This method is useful for scenarios such as:
- **Login screens:** After a user successfully logs in, you use `pushReplacement()` to navigate to the home screen
- **Splash screens:** A splash screen should only be visible once. After the app loads, it is replaced by the main content screen

Syntax of pushReplacement?

- You can use pushReplacement() with a MaterialPageRoute

```
Navigator.pushReplacement(  
  context,  
  MaterialPageRoute(builder: (context) => NewScreen()),  
);
```

- Alternatively, for named routes, you can use the pushReplacementNamed() method

```
Navigator.pushReplacementNamed(context, '/newScreen');
```